Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build hash and anchor certificate evidence for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: register(digest) commits a SHA-256 evidence digest; issuer authorization and duplicate rejection define acceptance. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Review prompt

Review every external function for authorization, state transitions, integer units and invariants. Point to exact lines. Produce a requirement-to-assertion matrix and add a failing test before repairing any defect.

Scenario-specific generation constraints

Read the synthetic certificate record. Canonicalize JSON with sorted keys and compact separators, hash with SHA-256 and call register(digest) as the issuer. Read registered(digest) before/after. Attempt the identical digest as issuer and a forged digest as outsider. Both calls must revert.

Expected assertions: registered(digest)=true; duplicate and unauthorized writes rejected.

Extension prompt

Change one record field and prove its digest differs; register the revised evidence as issuer. Return the exact changed source lines and positive/negative tests.